ChatApp | Security Findings & Remediation

Purpose & Scope: **2**
1. NoSQL Injection 4
2. JWT Misconfiguration / Auth Bypass 5
3. CSRF (Cross-Site Request Forgery) 6
4. XSS (Cross-Site Scripting) 7
5. File Upload / RCE 8
6. SSRF (Server-Side Request Forgery) 9
7. WebSocket Session Hijacking 10

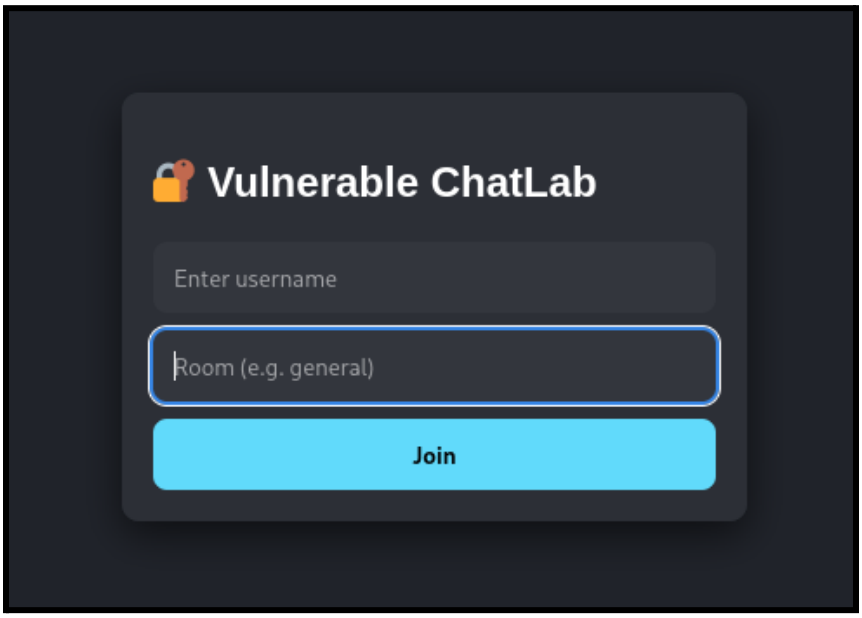
Conclusion: **11**

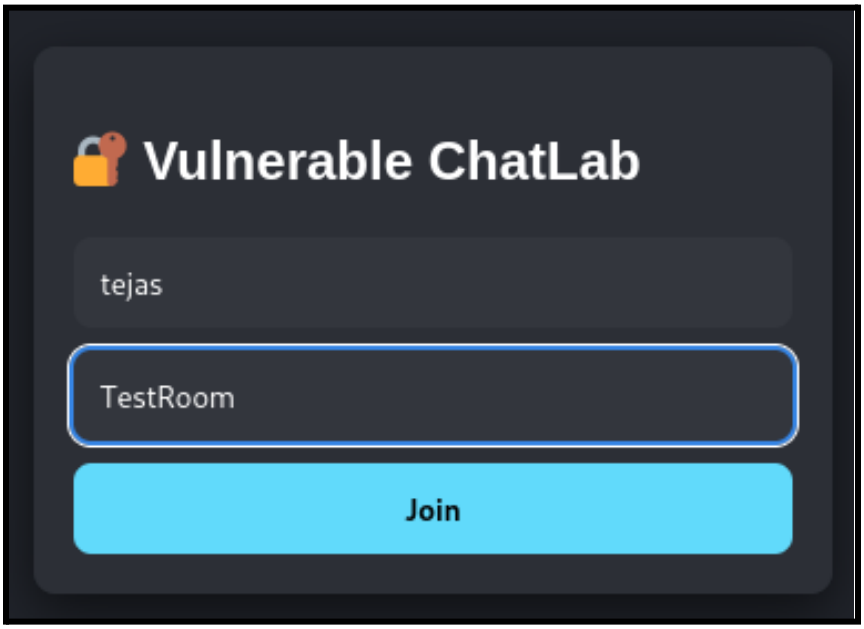
ChatApp | Security Findings & Remediation

Purpose & Scope:

Purpose: Educational demonstration of common Web Application Penetration Testing (WAPT) vulnerabilities on the [Chat-App] (Branch: **master**) ReactJS application.
Scope: Application intentionally contains insecure code/configurations for academic purposes.
Use Case:

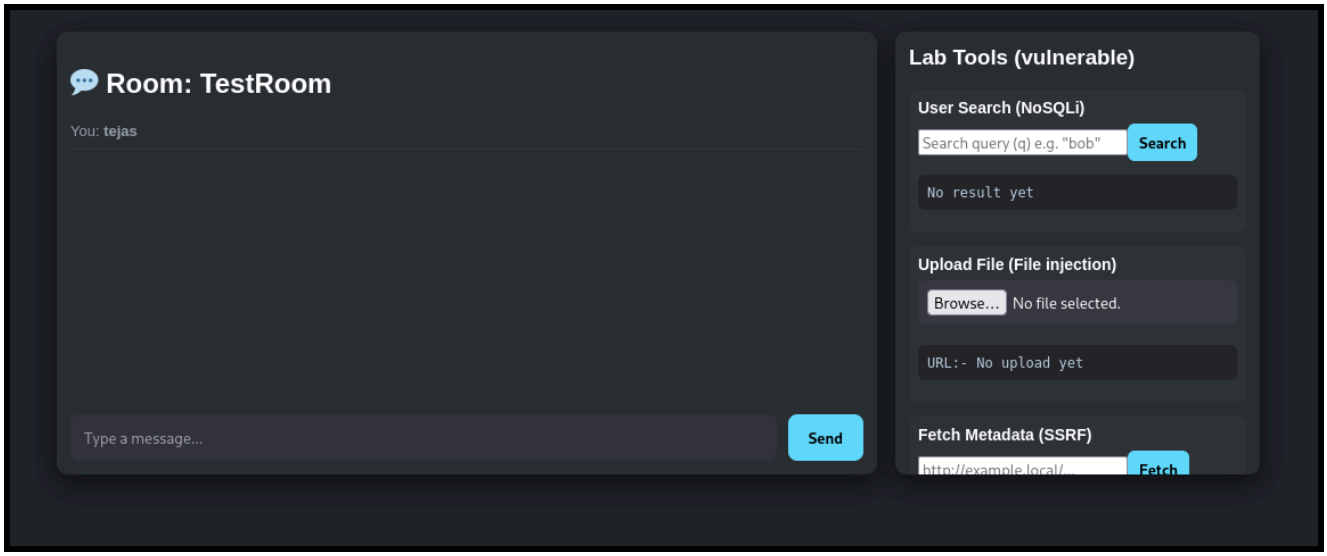
- User logs in with username & room ID:



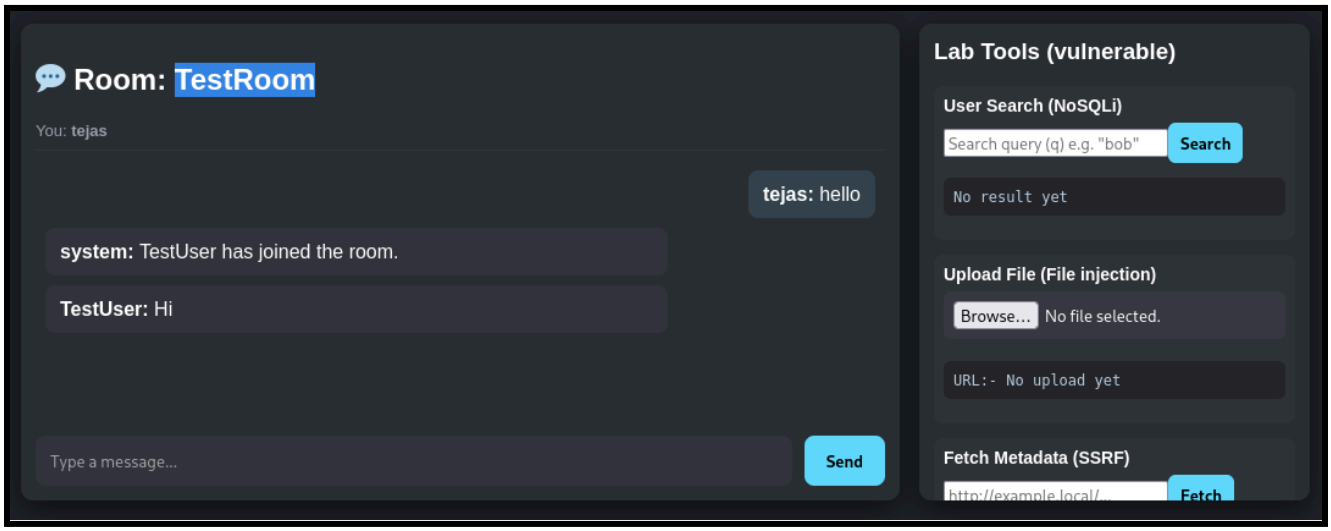


- Application allows chatting, file upload, metadata fetch, and other features:

ChatApp | Security Findings & Remediation



- Inside these features, multiple vulnerabilities can be demonstrated:



ChatApp | Security Findings & Remediation

1. NoSQL Injection

Description:
The application takes user input (q) and directly parses it into MongoDB queries using JSON.parse(). In lab mode, this bypasses all sanitization, enabling injection of operators like **$gt, $ne, $or.**

Real-World Context:

MongoDB and other NoSQL databases don't use SQL syntax, but they are equally vulnerable to injection. Attacks like **NoSQLi** have been used in account takeover incidents.

Affected Code:

```
// 5) NoSQL injection demo (vulnerable search; constructs query with user input)
app.get('/api/users/search', async (req, res) => {
  const q = req.query.q || '';
  // INTENTIONALLY VULNERABLE: building a query object directly from user input in lab mode
  // For example, a user could provide JSON that manipulates operators.
  let query;
  try {
    if (LAB_MODE) {
      // LAB: try to parse as JSON (vulnerable if user sends operator fields)
      query = JSON.parse(q);
    } else {
      query = { username: { $regex: q, $options: 'i' } };
    }
  } catch (e) {
    query = { username: { $regex: q, $options: 'i' } };
  }
  const results = await db.collection('users').find(query).limit(50).toArray();
  res.json({ count: results.length, results });
});
```

Here, the vulnerable code is the JSON.parse(q) as it is directly executing the query into the database.

Exploitation:

- Attacker supplies payload:
  {"$ne": null} or username ||**'1'=='1'** → Returns all users.
- Possible account enumeration or bypassing filters.

Impact:

- Account enumeration.
- Authentication bypass if the login logic is vulnerable.
- Potential full data leakage.

Remediation:

- Enforce strict input types (only strings).

- Disallow raw JSON parsing from the client.
- Use parameterized queries / ODM features (Mongoose).

2. JWT Misconfiguration / Auth Bypass

Description:

In the application, signature validation is not enforced for tokens with alg: none.

Real-World Context:

- In 2015, Auth0 disclosed widespread abuse of the **"alg": "none"** bug in libraries.
- Weak secrets also caused real breaches (e.g., guessing JWT signing key).

Affected Code:

```
function verifyJwt(token) {
  try {
    const decoded = jwt.decode(token, { complete: true }); // decode without verification
    // UNSAFE: if the JWT header specifies "alg":"none", accept the payload directly
    if (LAB_MODE && decoded.header.alg === 'none') {
      return decoded.payload; // bypass signature check
    }
    return jwt.verify(token, JWT_SECRET); // normal verification
  } catch (e) {
    return null;
  }
}
```

Exploitation:

- Attacker forges a token with no signature **("alg":"none")** or uses the bypass token to impersonate users.

Impact:

- Authentication bypass.
- Privilege escalation to admin.
- Data exfiltration.

Remediation:

- Never accept **alg=none.**
- Rotate and securely store secrets.

- Enforce algorithm whitelist:

jwt.verify(token, secret, { algorithms: ['HS256'] });

3. CSRF (Cross-Site Request Forgery)

Description:
Endpoints like /api/rooms/create use only cookie authentication, no CSRF protection.

Real-World Context:
CSRF has been abused in:

- **Gmail 2007 CSRF attack**, where emails could be forwarded without user consent.
- Bank transfers are initiated silently via hidden forms.

Affected Code:

```
// 2) CSRF-vulnerable room creation (uses cookie auth and lacks CSRF protections in LAB_MODE)
app.post('/api/rooms/create', (req, res) => {
  // NOTE: This endpoint uses cookie-based auth and does NOT check CSRF in lab mode
  const token = req.cookies.token;
  const user = verifyJwt(token);
  if (!user) return res.status(401).json({ error: 'unauthenticated' });

  const { room } = req.body;
  if (!room) return res.status(400).json({ error: 'room required' });

  // Simple room creation
  db.collection('rooms').insertOne({ room, owner: user.username, createdAt: new Date() })
    .then(r => res.json({ ok: true, id: r.insertedId }))
    .catch(err => res.status(500).json({ error: String(err) }));
});
```

Exploitation:

- Attacker hosts a malicious page that auto-submits a POST request to /api/rooms/create using the victim's cookies.

Impact:

- Unauthorized room creation.
- In real apps: fund transfers, password changes, or account deletion.

Remediation:

- Implement **CSRF tokens** (csurf middleware).
- Set **SameSite=Strict** on cookies.
- Validate Origin/Referer.

ChatApp | Security Findings & Remediation

4. XSS (Cross-Site Scripting)

Description:
Chat messages are rendered directly via **dangerouslySetInnerHTML**.

Real-World Context:
XSS has been used in:

- **MySpace Samy Worm (2005):** self-propagating XSS gained 1M friends overnight.
- Stolen session cookies in banking portals.

Affected Code:

```
<div className="messages" id="messages">
  {messages.map((m, i) => (
    <div key={i} className={`message ${m.user === "system" ? "system" : (m.username === username ? "self" : "")}`}>
      <strong>{m.username || m.user}: </strong>
      {/* keep vulnerable render for lab */}
      <span dangerouslySetInnerHTML={{ __html: m.text }} />
    </div>
  ))}
</div>
```

Here, the application is directly setting the HTML contents to the DOM.

Exploitation:

- Payload: **<script>alert('XSS')</script>**
- Executes arbitrary JavaScript in the victim's browser (**session theft, keylogging**, etc.).

Impact:

- Session hijacking.
- WebSocket hijack (via stolen JWT).
- Defacement or phishing inside chat.

Remediation:

- Encode output before rendering.
- Content Security Policy (CSP).
- Use safe frameworks/templates.

ChatApp | Security Findings & Remediation

5. File Upload / RCE

Description:
File upload endpoint stores files directly in **/uploads/** without validation.

Real-World Context:

- File upload vulnerabilities often lead to **Remote Code Execution (RCE)**.
- Famous cases: malicious PHP shells uploaded via profile picture features.

Affected Code:

```
// 3) File upload (lab: stores files in webroot and serves them directly → FILE INJECTION demo)
const upload = multer({ dest: './uploads/' });
app.post('/api/upload', upload.single('file'), (req, res) => {
  // Vulnerable: stores uploaded files in uploads/ which is served directly via static
  // In production, you must validate filename, disallow executable perms, and store outside webroot.
  console.log('UPLOAD', req.file);
  res.json({ filename: req.file.filename, originalname: req.file.originalname, mimetype: req.file.mimetype, acce
});
```

Exploitation:

- The attacker uploads a .js, .php, or .exe file.
- If served from /uploads/, the attacker can execute or trick users into downloading malicious files.

Impact:

- Full server compromise.
- Malware distribution.

Remediation:

- Accept only specific MIME types (jpg/png/pdf).
- Rename files with UUIDs.
- Store outside web root.

ChatApp | Security Findings & Remediation

6. SSRF (Server-Side Request Forgery)

Description:
/api/fetchmeta fetches arbitrary URLs from user input.

Real-World Context:

- **Capital One breach (2019):** exploited SSRF in AWS metadata service.
- SSRF can bypass firewalls, access internal databases, and cloud services.

Affected Code:

```
app.post('/api/fetchmeta', async (req, res) => {
  const { url } = req.body;
  if (!url) return res.status(400).json({ error: 'url required' });

  // Resolve hostname to IP and log it (demonstrates SSRF risk)
  try {
    const u = new URL(url);
    const ips = await dns.lookup(u.hostname, { all: true });
    console.log('RESOLVED IPS', u.hostname, ips);
    // LAB: no allowlist enforced here (vulnerable). In real app, check allowlist & block private ranges.
    const resp = await fetch(url, { method: 'GET', timeout: 5000 });
    const contentType = resp.headers.get('content-type') || '';
    const contentLength = resp.headers.get('content-length') || 'unknown';
    return res.json({ ok: true, status: resp.status, contentType, contentLength });
  } catch (e) {
    return res.status(500).json({ error: String(e) });
  }
});
```

Exploitation:

- Attacker supplies: http://127.0.0.1:27017
- Server attempts to fetch internal resources.

Impact:

- Internal network mapping.
- Cloud credential theft.

Remediation:

- Allowlist trusted domains.
- Block private IPs (10.x, 127.x, 169.254.x).
- Use SSRF-safe libraries with URL parsing.
- Use short timeouts and response size limits.

ChatApp | Security Findings & Remediation

7. WebSocket Session Hijacking

Description

Intercepting the WebSocket handshake and manipulating the messages.

Real-World Context:

- If combined with XSS, an attacker can steal tokens and hijack sessions.
- Similar to real Slack/Discord-style attacks.

Affected Code:

```
// SEND handler - store & broadcast
socket.on('send', (data) ⇒ {
  // data: { room, text }
  if (!data || !data.room) return;
  const username = socket.username || (socket.user ? socket.user.username : (socket.labUnauth ? 'lab-unauth' : 'anon'));
  db.collection('messages').insertOne({ room: data.room, text: data.text, username, ts: new Date() })
    .catch(err ⇒ console.error('DB insert error', err));
  console.log('Received send:', data);
  io.to(data.room).emit('message', { room: data.room, text: data.text, username });
  console.log("Emitting message to room:", data.room, data.text);
});
```

Exploitation:

- An attacker with XSS steals the token and connects to the WebSocket as the victim.

Impact:

- Full account takeover.
- Live eavesdropping in chats.

Remediation:

- Store tokens in httpOnly cookies.
- Validate JWT during WebSocket handshake.
- Token rotation.

Conclusion:

This intentionally vulnerable chat application demonstrates **seven OWASP Top-10 style vulnerabilities** with real attack potential. Each is mapped to real-world incidents to show the seriousness.